

YAML文件基础定义与用途

- 后缀与用途：YAML文件的常用后缀为 `.yaml` 或 `.yml`，最主要的用途是作为项目的配置文件。
- AI领域的应用场景：人工智能领域训练模型时，需要用YAML配置模型参数。CV方向做目标检测时，YOLO系列框架中包含大量YAML配置文件，用于配置数据集和网络结构。
- 基础语法规则：YAML采用键值对结构存储数据，格式为 `key: 空格值`，这种结构对应Python中的字典类型。

YAML可存储的数据类型

- 纯量类型存储规则：YAML支持字符串、布尔、整数、浮、空、日期、时间等纯量类型。字符串不需要添加单引号或双引号，直接书写即可。
- 列表类型存储规则：YAML中用开头带横杠的换行写法表示列表，对应Python中的列表，列表内可存储任意类型的数据。
- 嵌套结构支持：YAML支持嵌套结构，列表中可以嵌套字典，字典中也可以嵌套其他字典或列表，写法灵活。

YOLO框架中YAML文件的实际应用

- 数据集配置文件：YOLO框架的 `data` 文件夹下全部是YAML文件，用于配置数据集，包含训练集、验证集、测试集的路径，以及检测类别名称。
- 网络结构配置文件：YOLO框架的 `models` 文件夹下的YAML文件，用于配置网络结构，不同尺寸的模型（对应类似服装尺码n、s、m、l、x）有不同的缩放系数和参数量配置。
- 数据集拆分逻辑：使用YOLO训练模型时，需要将总数据集拆分为训练集、验证集/测试集，训练集用于调整模型参数，测试集用于评估模型泛化能力，一般项目拆分为训练集+测试集两部分即可。

```
In [5]: # YAML 文件解析示例
# 导入 yaml 库用于解析 YAML 格式的文件
import yaml
# pprint 可以让打印的字典/列表更美观
from pprint import pprint

# =====
# yaml.safe_load() 的作用:
#   读取 YAML 文件内容,自动转换成 Python 的字典/列表等数据类型
#   safe_load 比 load 更安全,不会执行 YAML 中的任意代码
# =====

# 指定要读取的 YAML 文件名(和本脚本在同一目录下)
yaml_file = "example.yaml"

print("=" * 60)
print("📖 读取 YAML 文件")
print("=" * 60)

# 1. 打开文件('r' 表示只读模式,encoding='utf-8' 支持中文)
# 2. yaml.safe_load(f) 自动解析整个文件
```

```

# 3. 返回的数据 data 是一个 Python 字典
with open(yaml_file, "r", encoding="utf-8") as f:
    data = yaml.safe_load(f)

# -----
# 第一部分:基础数据类型
# 说明:YAML 会自动识别值的类型(字符串/数字/布尔/空值)
# -----

# 1. 基本键值对
# YAML 的 key: value 直接映射为 Python 字典的键值对
print("\n1 基本键值对:")
print(f"    name: {data['name']} (类型: {type(data['name']).__name__})")
print(f"    age: {data['age']} (类型: {type(data['age']).__name__})")
print(f"    is_student: {data['is_student']} (类型: {type(data['is_student']).__name__})")

# 2. 字符串
# YAML 中用 | 表示多行字符串,会保留换行符
print("\n2 字符串:")
print(f"    greeting: {data['greeting']}")
print(f"    description: {repr(data['description'])}")

# 3. 数字类型
# YAML 自动识别:整数,浮点数,十六进制,科学计数法
print("\n3 数字类型:")
print(f"    integer_value: {data['integer_value']} ({type(data['integer_value']).__name__})")
print(f"    float_value: {data['float_value']} ({type(data['float_value']).__name__})")
# 0xFF 是十六进制,YAML 自动解析为十进制整数 255
print(f"    hex_value: {data['hex_value']} ({type(data['hex_value']).__name__})")
# 1.23e+4 是科学计数法,表示 1.23 x 10^4 = 12300
print(f"    scientific: {data['scientific']} ({type(data['scientific']).__name__})")

# 4. 布尔值
# YAML 支持 true/false,yes/no 等多种写法
print("\n4 布尔值:")
print(f"    active: {data['active']} ({type(data['active']).__name__})")

# -----
# 第二部分:容器类型(列表和字典)
# -----

# 5. 列表(数组)
# YAML 中以 - 开头的行表示列表项
print("\n5 列表:")
print(f"    fruits: {data['fruits']} (共 {len(data['fruits'])} 项)")
# 遍历列表并打印每一项
for i, fruit in enumerate(data['fruits'], 1):
    print(f"        {i}. {fruit}")

# 6. 嵌套对象(字典套字典)
# YAML 通过缩进来表示层级关系
print("\n6 嵌套对象 (person):")
person = data['person'] # person 是一个字典
print(f"    姓名: {person['name']}")
print(f"    年龄: {person['age']}")
# address 也是一个字典,继续用 .get() 或 [] 取值
print(f"    地址: {person['address']['street']},{person['address']['city']}")

# 7. 对象列表(列表中包含多个字典)
# 非常常见的数据结构,比如员工列表,商品列表等

```

```

print("\n7 对象列表 (employees):")
print(f"    {'姓名':<6} {'部门':<8} {'薪资':<8}")
print(f"    {'-'*22}")
# 遍历每个员工字典,取出对应的字段
for emp in data['employees']:
    print(f"    {emp['name']:<6} {emp['department']:<8} {emp['salary']:<8,}")

# -----
# 第三部分:高级特性
# -----

# 8. 锚点引用(& 和 <<: *)
# &defaults 定义默认值,<<: *defaults 表示"合并到此处"
# yaml.safe_load 会自动展开合并,就像复制了一份默认值
print("\n8 锚点引用:")
print(f"    server1: {data['server1']}")
print(f"    server2: {data['server2']} (注意 timeout 被覆盖为 60)")

# 9. 日期和时间
# YAML 自动将 ISO 8601 格式的字符串解析为 Python 的 date/datetime 对象
print("\n9 日期时间:")
print(f"    start_date: {data['start_date']} ({type(data['start_date']).__name__})")
print(f"    meeting_time: {data['meeting_time']} ({type(data['meeting_time']).__name__})")

# -----
# 第四部分:打印完整的数据结构总览
# 展示每个顶层键对应的 Python 数据类型和内容摘要
# -----

print("\n" + "=" * 60)
print(f"📊 数据结构总览")
print("=" * 60)
# 遍历 data 字典的所有键值对
for key, value in data.items():
    val_type = type(value).__name__ # 获取 Python 类型名称
    # 根据不同类型生成更友好的摘要说明
    if isinstance(value, list):
        summary = f"列表[{len(value)}项]"
    elif isinstance(value, dict):
        summary = f"字典[{len(value)}个键]"
    elif isinstance(value, str):
        summary = f"字符串({len(value)}字符)"
    elif isinstance(value, bool):
        summary = f"布尔值({value})"
    elif value is None:
        summary = "空值(None)"
    else:
        summary = str(value)
    print(f"    {key:<20} [{val_type:<8}] {summary}")

```

=====

 读取 YAML 文件

=====

- 1 基本键值对：
name: 张三 (类型: str)
age: 25 (类型: int)
is_student: False (类型: bool)
- 2 字符串：
greeting: Hello, World!
description: '这是一个多行字符串。 \n第二行内容。 \n第三行内容。 \n'
- 3 数字类型：
integer_value: 42 (int)
float_value: 3.14 (float)
hex_value: 255 (int)
scientific: 12300.0 (float)
- 4 布尔值：
active: True (bool)
- 5 列表：
fruits: ['苹果', '香蕉', '橙子', '葡萄'] (共 4 项)
1. 苹果
2. 香蕉
3. 橙子
4. 葡萄
- 6 嵌套对象 (person):
姓名: 李四
年龄: 30
地址: 长安街,北京
- 7 对象列表 (employees):

姓名	部门	薪资
小王	技术部	15,000
小李	市场部	12,000
小张	人事部	13,000
- 8 锚点引用：
server1: {'timeout': 30, 'retries': 3, 'debug': False, 'host': 'server1.example.com'}
server2: {'timeout': 60, 'retries': 3, 'debug': False, 'host': 'server2.example.com'} (注意 timeout 被覆盖为 60)
- 9 日期时间：
start_date: 2024-01-01 (date)
meeting_time: 2024-06-15 14:30:00+08:00 (datetime)

=====

 数据结构总览

=====

name	[str	] 字符串(2字符)
age	[int	] 25
city	[str	] 字符串(2字符)
is_student	[bool	] 布尔值(False)
greeting	[str	] 字符串(13字符)
description	[str	] 字符串(25字符)

integer_value	[int]	42
float_value	[float]	3.14
hex_value	[int]	255
scientific	[float]	12300.0
active	[bool]	布尔值(True)
completed	[bool]	布尔值(False)
enabled	[bool]	布尔值(True)
disabled	[bool]	布尔值(False)
empty_field	[NoneType]	空值(None)
another_null	[NoneType]	空值(None)
fruits	[list]	列表[4项]
colors	[list]	列表[3项]
person	[dict]	字典[3个键]
employees	[list]	列表[3项]
default_settings	[dict]	字典[3个键]
server1	[dict]	字典[4个键]
server2	[dict]	字典[4个键]
database	[dict]	字典[4个键]
start_date	[date]	2024-01-01
meeting_time	[datetime]	2024-06-15 14:30:00+08:00

线程串行执行的逻辑

- 串行的核心特征：一个线程内只能同时执行1个任务，多个任务必须按顺序逐个执行。
- 串行的类比解释：串行可以类比为糖葫芦串，需要从一端开始逐个啃食，不能同时从多个位置啃食。
- 串行下载的效果：单线程串行下载多个文件时，必须等待前一个文件下载完成才能开始下一个，整体下载速度更慢。

线程并行（多线程）执行的逻辑

- 并行的核心特征：一个进程可以开启多个线程，多个线程可以同时执行不同的任务。
- 并发并行的实现差异：只有多核CPU才能实现真正的多线程同时执行；单核CPU依靠CPU高速切换线程，给用户制造同时执行的假象，整体速度依然比单线程更快。
- 多线程的核心优势：多线程可以更高效利用CPU、内存等系统资源，适当提升程序的执行效率。

多线程的优缺点与数量误区

- 线程不是越多越好：线程数量过多会增加CPU调度开销，单个线程被CPU调度到的概率会降低，100个线程单个线程被调度概率仅为1%，部分线程可能长期得不到执行。
- 多线程的优点：适当数量的多线程可以适当提高程序执行效率，提升系统资源利用率。
- 多线程的缺点：每个开启的线程都会占用内存空间，大量线程会占用大量内存降低程序性能；线程越多，线程之间的数据共享、通信问题越复杂，程序设计难度会提升。

Python主线程的验证方法

- 主线程的默认特性：Python程序运行时默认会自动开启1个名为 `MainThread` 的主线程。
- 获取当前线程的代码方法：导入 `threading` 模块后，调用 `threading.current_thread()` 即可获取当前执行的线程对象。

- 线程属性的操作：可以通过 `线程对象.name` 获取线程名称，也可以直接修改该属性自定义线程名；通过 `线程对象.ident` 可以获取线程编号，线程编号每次启动程序都会随机分配，不会固定。

Python创建多线程的第一种方法

- 创建流程说明：第一步先定义要执行任务的函数，第二步通过 `threading.Thread` 类创建线程对象，第三步调用线程对象的 `start()` 方法启动线程。
- 参数绑定规则：创建线程对象时，通过 `target` 参数绑定要执行的函数，绑定仅需要写函数名，不需要加小括号执行；通过 `name` 参数可以自定义新建线程的名称，默认名称为 `Thread-1`、`Thread-2` 依次递增。
- 启动规则说明：仅创建线程对象不调用 `start()` 方法，线程不会执行；每个线程对象的 `start()` 方法最多只能调用1次，不能重复启动同一个线程。

Python创建多线程的第二种方法

- 创建流程说明：自定义类继承 `threading.Thread` 父类，重写父类的 `run()` 方法，将需要执行的任务写在 `run()` 方法中，创建子类对象后调用 `start()` 方法即可启动线程。
- 继承的核心要求：继承 `threading.Thread` 后，需要调用 `super().__init__()` 初始化父类的属性，必须重写 `run()` 方法存放执行任务。
- 执行逻辑说明：不管使用哪种方式创建线程，启动线程都调用 `start()` 方法，最终线程执行的任务都会进入 `run()` 方法中运行。

```
In [6]: # =====
# Python 多线程示例 (threading)
# 线程: 程序中的"小工人", 可以同时做多件事
# 比如: 一边下载文件, 一边处理用户输入
# =====
import threading
import time
import random

# -----
# 第一步: 定义一个"任务"函数
# 这个函数就是每个线程要执行的工作
# -----
def worker(name, work_time):
    """
    模拟一个工作线程

    参数:
        name: 线程的名字
        work_time: 工作需要的时间(秒)
    """
    print(f"🧑 [{name}] 开始工作, 预计需要 {work_time} 秒...")

    # 模拟"工作中"--每隔 1 秒打印一次进度
    for i in range(work_time):
        time.sleep(1) # sleep(1) = 暂停 1 秒
        print(f"  [{name}] 工作中... 已完成 {i+1}/{work_time} 秒")

    print(f"✅ [{name}] 工作完成! 耗时 {work_time} 秒")
```

```

# -----
# 第二步: 创建并启动多个线程
# target=worker    -> 指定线程要执行的函数
# args=(...)       -> 传给函数的参数
# -----
print("=" * 60)
print("🚀 单线程模式(按顺序执行)")
print("=" * 60)

# 先看单线程的效果: 一个接一个执行, 总共需要 3+2+4 = 9 秒
start = time.time()
worker("张三", 3)
worker("李四", 2)
worker("王五", 4)
end = time.time()
print(f"\n🕒 单线程总耗时: {end - start:.1f} 秒\n")

# -----
# 第三步: 多线程模式
# 多个线程"同时"运行, 互不干扰
# 总耗时 ≈ 最慢的线程的耗时(而不是加起来)
# -----
print("=" * 60)
print("🔥 多线程模式(同时执行)")
print("=" * 60)

# 创建 3 个线程对象
threads = []
names = ["张三", "李四", "王五"]
times = [3, 2, 4]

for name, t in zip(names, times):
    # threading.Thread(target=函数, args=(参数,))
    # 注意: args 必须是一个元组, 单个参数要加逗号 (n,)
    thread = threading.Thread(target=worker, args=(name, t))
    threads.append(thread)

# 启动所有线程(这时它们开始"同时"运行)
start = time.time()
print("\n>>> 启动所有线程...\n")
for t in threads:
    t.start() # start() 让线程开始执行

# 等待所有线程执行完毕
# join() 意思是"主线程在这里等待, 直到子线程完成"
print("\n>>> 等待所有线程结束...\n")
for t in threads:
    t.join() # join() 会阻塞, 直到该线程结束

end = time.time()
print(f"\n🕒 多线程总耗时: {end - start:.1f} 秒")
print("🚀 多线程让多个任务同时进行, 大大节省了总时间!")

# -----
# 第四步: 线程安全 -- 多个线程同时修改同一变量会出问题
# 演示用锁 (Lock) 来保护共享资源
# -----
print("\n" + "=" * 60)
print("🔒 线程安全示例(用 Lock 保护共享变量)")
print("=" * 60)

```

```

# 共享计数器(多个线程同时修改它)
counter = 0
# 创建一个锁对象
lock = threading.Lock()

def unsafe_increment():
    """不使用锁 -- 可能出现数据错乱"""
    global counter
    temp = counter
    time.sleep(0.1) # 故意暂停,增加线程切换的概率
    counter = temp + 1

def safe_increment():
    """使用锁 -- 保证同一时刻只有一个线程修改"""
    global counter
    with lock: # 获取锁,其他线程必须等待
        temp = counter
        time.sleep(0.1)
        counter = temp + 1
    # 离开 with 块自动释放锁

# 测试不加锁的情况
counter = 0
threads = [threading.Thread(target=unsafe_increment) for _ in range(10)]
for t in threads: t.start()
for t in threads: t.join()
print(f"❌ 不加锁: counter = {counter}(期望 10,但往往小于 10)")

# 测试加锁的情况
counter = 0
threads = [threading.Thread(target=safe_increment) for _ in range(10)]
for t in threads: t.start()
for t in threads: t.join()
print(f"✅ 加锁: counter = {counter}(总是 10)")

# -----
# 小总结
# -----
print("\n" + "=" * 60)
print("📁 多线程小结")
print("=" * 60)
print("""
    threading.Thread(target=函数, args=(参数,)) # 创建线程
    thread.start() # 启动线程
    thread.join() # 等待线程结束

    threading.Lock() # 创建锁
    with lock: ... # 保护共享资源(自动获取和释放锁)
""")

```

🚀 单线程模式(按顺序执行)

```

=====
👤 [张三] 开始工作, 预计需要 3 秒 ...
[张三] 工作中... 已完成 1/3 秒
[张三] 工作中... 已完成 2/3 秒
[张三] 工作中... 已完成 3/3 秒
✅ [张三] 工作完成! 耗时 3 秒
👤 [李四] 开始工作, 预计需要 2 秒 ...
[李四] 工作中... 已完成 1/2 秒
[李四] 工作中... 已完成 2/2 秒
✅ [李四] 工作完成! 耗时 2 秒
👤 [王五] 开始工作, 预计需要 4 秒 ...
[王五] 工作中... 已完成 1/4 秒
[王五] 工作中... 已完成 2/4 秒
[王五] 工作中... 已完成 3/4 秒
[王五] 工作中... 已完成 4/4 秒
✅ [王五] 工作完成! 耗时 4 秒
  
```

🕒 单线程总耗时: 9.0 秒

🔥 多线程模式(同时执行)

>>> 启动所有线程...

```

👤 [张三] 开始工作, 预计需要 3 秒...
👤 [李四] 开始工作, 预计需要 2 秒...
👤 [王五] 开始工作, 预计需要 4 秒...
  
```

>>> 等待所有线程结束...

```

[张三] 工作中... 已完成 1/3 秒
[王五] 工作中... 已完成 1/4 秒
[李四] 工作中... 已完成 1/2 秒
[王五] 工作中... 已完成 2/4 秒   [张三] 工作中... 已完成 2/3 秒
[李四] 工作中... 已完成 2/2 秒
✅ [李四] 工作完成! 耗时 2 秒

[张三] 工作中... 已完成 3/3 秒   [王五] 工作中... 已完成 3/4 秒

✅ [张三] 工作完成! 耗时 3 秒
[王五] 工作中... 已完成 4/4 秒
✅ [王五] 工作完成! 耗时 4 秒
  
```

🕒 多线程总耗时: 4.0 秒

🎉 多线程让多个任务同时进行, 大大节省了总时间!

🔒 线程安全示例(用 Lock 保护共享变量)

```

❌ 不加锁: counter = 1(期望 10, 但往往小于 10)
✅ 加锁: counter = 10(总是 10)
  
```

📄 多线程小结

```
threading.Thread(target=函数, args=(参数,)) # 创建线程
```

```

thread.start()      # 启动线程
thread.join()       # 等待线程结束

threading.Lock()    # 创建锁
with lock: ...      # 保护共享资源(自动获取和释放锁)

```

```

In [9]: # =====
# 继承 threading.Thread 方式创建线程
# 一个线程输出数字, 另一个线程输出字母
# =====
import threading
import time

# -----
# 第一步: 定义数字线程类
# 继承 threading.Thread, 重写 run() 方法
# -----
class NumberThread(threading.Thread):
    """输出数字的线程"""

    def run(self):
        """
        线程启动后自动执行的方法
        输出 1~10 的数字
        """
        print("\033[34m [数字线程] 开始输出数字:")
        for i in range(1, 11):
            print(f"\033[34m {i}")
            time.sleep(0.2) # 让线程切换更明显
        print("\033[34m [数字线程] 输出完毕!")

# -----
# 第二步: 定义字母线程类
# 同样继承 threading.Thread, 重写 run()
# -----
class LetterThread(threading.Thread):
    """输出字母的线程"""

    def run(self):
        """输出 A~J 的字母"""
        print("\033[32m [字母线程] 开始输出字母:")
        for ch in "ABCDEFGHIJ":
            print(f"\033[32m {ch}")
            time.sleep(0.2)
        print("\033[32m [字母线程] 输出完毕!")

# -----
# 第三步: 创建线程对象并启动
# -----
print("\033[31m 🚦 两个线程同时运行:")
print("    数字线程输出 1~10, 字母线程输出 A~J")
print("    它们会交替执行, 看起来像\"同时\"进行\n')

# 创建两个线程对象
t1 = NumberThread() # 数字线程
t2 = LetterThread() # 字母线程

# 启动线程(start() 会自动调用 run() 方法)
t1.start()

```

```

t2.start()

# 等待两个线程都执行完毕
t1.join()
t2.join()

print("\n🚩 两个线程都执行完毕!")

# =====
# 扩展: 传参方式 -- 继承 + __init__
# 自定义初始化的线程类
# ⚠️ 注意: 属性名不能与 Thread 类的方法名冲突(如 start)
# =====
print("\n" + "=" * 60)
print("🚩 带参数的继承方式(自定义 begin/end)")
print("=" * 60)

class RangeThread(threading.Thread):
    """可指定范围的数字线程"""

    def __init__(self, name, begin, end):
        """
        参数:
            name: 线程名称
            begin: 起始值
            end: 结束值
        """
        # 先调用父类的 __init__()
        super().__init__()
        self.thread_name = name
        self.begin = begin
        self.end = end

    def run(self):
        print(f"🚩 [{self.thread_name}] 输出 {self.begin} ~ {self.end}:")
        for i in range(self.begin, self.end + 1):
            print(f"    [{self.thread_name}] {i}")
            time.sleep(0.15)
        print(f"🚩 [{self.thread_name}] 完成!")

# 创建两个不同范围的线程
t3 = RangeThread("线程A", 1, 5)
t4 = RangeThread("线程B", 10, 15)

t3.start()
t4.start()
t3.join()
t4.join()

print("\n✅ 带参数的继承线程执行完毕!")

```

🚩 两个线程同时运行：
 数字线程输出 1~10,字母线程输出 A~J
 它们会交替执行,看起来像"同时"进行

1 2 [数字线程] 开始输出数字:

1

abc [字母线程] 开始输出字母:

A

2

B

3 C

D

4

5 E

6

F

G

7

H

8

I

9

J

10

abc [字母线程] 输出完毕!

1 2 3 4 [数字线程] 输出完毕!

🎉 两个线程都执行完毕!

=====

🚩 带参数的继承方式(自定义 begin/end)

=====

🚩 [线程A] 输出 1 ~ 5:

[线程A] 1

🚩 [线程B] 输出 10 ~ 15:

[线程B] 10

[线程B] 11

[线程A] 2

[线程B] 12

[线程A] 3

[线程A] 4

[线程B] 13

[线程A] 5 [线程B] 14

[线程B] 15

🚩 [线程A] 完成!

🚩 [线程B] 完成!

✅ 带参数的继承线程执行完毕!

线程创建与启动步骤

- 导入语法简化: 可以使用 `from threading import Thread` 的导入方式, 后续直接使用Thread创建对象, 不需要重复写threading前缀。
- 绑定任务规则: 创建线程对象给target参数传值时, 只需要绑定函数名, 不能直接调用函数。

- 主线程输出说明：创建并启动两个子线程后，在主线程末尾打印“主线程结束”，演示主线程和子线程的执行关系。

多线程执行特性演示

- 程序终止规则：主线程代码执行完成后，程序不会立即终止，会等待所有子线程执行完成后才会终止，因此“主线程结束”的输出可能出现在所有输出的任意位置。
- 调度随机性：线程的执行顺序由CPU调度决定，哪个线程被调度就执行哪个，因此输出的先后顺序不确定，每次运行结果都不相同。
- 输出混乱原因：print函数的执行分为两步，先打印内容再输出换行符，如果打印内容完成后还没输出换行符时，CPU就被调度给其他线程，就会出现多个线程内容输出在同一行的混乱问题。

执行速度调整操作

- 调整方法：导入time模块，在每个输出语句后添加 `time.sleep(0.5)`，让线程每次输出后休息0.5秒，更方便观察输出的混乱过程。
- 演示效果：休息会让出CPU，放大了输出混乱的效果，能更清晰看到多个线程交叉输出的问题。

输出混乱问题复现

- 拆分打印场景：如果将一行内容拆分为两次print打印，通过end参数修改换行规则，输出混乱问题会更明显，会出现字母和数字混排、内容归属错误的问题。
- 问题本质：该问题本质是多个线程共享输出资源，没有对临界代码进行保护，导致操作被其他线程打断。

线程锁解决混乱问题

- 锁的原理类比：线程锁类似于上厕所锁门，一个线程拿到锁后，其他线程无法进入执行，只有当前线程执行完成释放锁后，其他线程才能进入执行。
- 锁的创建方法：通过 `threading.Lock()` 创建锁对象，用变量lock接收即可，锁需要在函数外创建，所有线程共享同一把锁。
- 简化用法说明：不需要手动调用获取锁和释放锁方法，可以使用with语句管理锁，和管理文件句柄的用法一致，with代码块执行完成后会自动释放锁。
- 加锁范围规则：只需要给需要作为整体执行的打印代码加锁，sleep不需要加锁，sleep不影响输出内容的完整性，加锁后不会影响CPU的正常调度。

```
In [ ]: # =====
# 案例:线程安全问题 -- 卖票
# 需求:一共 50 张票,5 个窗口同时卖票
# 演示:不加锁 -> 数据错乱(超卖),加锁 -> 正确
# =====
import threading
import time

total_tickets = 50
lock = threading.Lock()

# -----
# 方案一:不加锁
# -----
```

```

def sell_tickets_unsafe(window_id):
    """卖票函数(不安全版本)"""
    global total_tickets

    while total_tickets > 0:
        time.sleep(0.01) # 模拟查票耗时

        if total_tickets > 0:
            current = total_tickets
            time.sleep(0.02) # 故意暂停, 让其他线程也读到同一张票号
            total_tickets = current - 1
            print(f" 🪟 窗口{window_id} 卖出一张票, 剩余 {total_tickets} 张")
        else:
            break

    print(f" 🪟 窗口{window_id} 没票了, 停止售票")

# -----
# 方案二: 加锁
# -----
def sell_tickets_safe(window_id):
    """卖票函数(安全版本)"""
    global total_tickets

    while True:
        with lock:
            if total_tickets <= 0:
                break
            current = total_tickets
            time.sleep(0.02)
            total_tickets = current - 1
            print(f" 🪟 窗口{window_id} 卖出一张票, 剩余 {total_tickets} 张")

        time.sleep(0.005)

    print(f" 🪟 窗口{window_id} 没票了, 停止售票")

# =====
# 测试不加锁
# =====
print("=" * 60)
print("❌ 不加锁版本 -- 看看会出什么问题")
print("=" * 60)

total_tickets = 50
threads = []
for i in range(1, 6):
    t = threading.Thread(target=sell_tickets_unsafe, args=(i,))
    threads.append(t)

for t in threads: t.start()
for t in threads: t.join()

print(f"\n🌟 结果: 最终剩余票数 = {total_tickets} (期望 0, 但出现负数 = 超卖!)\n")

# =====
# 测试加锁
# =====
print("=" * 60)
print("✅ 加锁版本 -- 数据安全可靠")

```

```

print("=" * 60)

total_tickets = 50
threads = []
for i in range(1, 6):
    t = threading.Thread(target=sell_tickets_safe, args=(i,))
    threads.append(t)

for t in threads: t.start()
for t in threads: t.join()

print(f"\n🎉 结果:最终剩余票数 = {total_tickets}(期望 0,完全正确!)")

# =====
# 总结
# =====
print("\n" + "=" * 60)
print("📄 总结")
print("=" * 60)
print("""
不加锁的后果:多个线程同时读到 total_tickets=1,
都认为还有票,结果都卖了出去,变成 -1,-2...
这就是典型的"超卖"问题。

解决办法:用 with lock: 把"查票->修改"包起来,
同一时刻只允许一个线程进入,保证数据安全。
""")

```

=====

✗ 不加锁版本 -- 看看会出什么问题

=====

窗口1	卖出一张票, 剩余	49 张
窗口2	卖出一张票, 剩余	49 张
窗口3	卖出一张票, 剩余	49 张
窗口4	卖出一张票, 剩余	49 张
窗口5	卖出一张票, 剩余	49 张
窗口1	卖出一张票, 剩余	48 张
窗口4	卖出一张票, 剩余	48 张
窗口3	卖出一张票, 剩余	48 张
窗口5	卖出一张票, 剩余	48 张
窗口2	卖出一张票, 剩余	48 张
窗口1	卖出一张票, 剩余	47 张
窗口3	卖出一张票, 剩余	47 张
窗口5	卖出一张票, 剩余	47 张
窗口4	卖出一张票, 剩余	47 张
窗口2	卖出一张票, 剩余	47 张
窗口3	卖出一张票, 剩余	46 张
窗口5	卖出一张票, 剩余	46 张
窗口1	卖出一张票, 剩余	46 张
窗口4	卖出一张票, 剩余	46 张
窗口2	卖出一张票, 剩余	46 张
窗口3	卖出一张票, 剩余	45 张
窗口5	卖出一张票, 剩余	45 张
窗口1	卖出一张票, 剩余	45 张
窗口2	卖出一张票, 剩余	45 张
窗口4	卖出一张票, 剩余	45 张
窗口5	卖出一张票, 剩余	44 张
窗口3	卖出一张票, 剩余	44 张
窗口1	卖出一张票, 剩余	44 张
窗口2	卖出一张票, 剩余	44 张
窗口4	卖出一张票, 剩余	44 张
窗口5	卖出一张票, 剩余	43 张
窗口1	卖出一张票, 剩余	43 张
窗口2	卖出一张票, 剩余	43 张
窗口4	卖出一张票, 剩余	43 张
窗口3	卖出一张票, 剩余	43 张
窗口5	卖出一张票, 剩余	42 张
窗口2	卖出一张票, 剩余	42 张
窗口1	卖出一张票, 剩余	42 张
窗口4	卖出一张票, 剩余	42 张
窗口3	卖出一张票, 剩余	42 张
窗口5	卖出一张票, 剩余	41 张
窗口4	卖出一张票, 剩余	41 张
窗口3	卖出一张票, 剩余	41 张
窗口2	卖出一张票, 剩余	41 张
窗口1	卖出一张票, 剩余	41 张
窗口5	卖出一张票, 剩余	40 张
窗口4	卖出一张票, 剩余	40 张
窗口3	卖出一张票, 剩余	40 张
窗口1	卖出一张票, 剩余	40 张
窗口2	卖出一张票, 剩余	40 张
窗口5	卖出一张票, 剩余	39 张
窗口1	卖出一张票, 剩余	39 张
窗口2	卖出一张票, 剩余	39 张
窗口3	卖出一张票, 剩余	39 张
窗口4	卖出一张票, 剩余	39 张
窗口5	卖出一张票, 剩余	38 张
窗口1	卖出一张票, 剩余	38 张

	窗口2	卖出一张票, 剩余	38 张	
	窗口3	卖出一张票, 剩余	38 张	
	窗口4	卖出一张票, 剩余	38 张	
	窗口5	卖出一张票, 剩余	37 张	
	窗口1	卖出一张票, 剩余	37 张	
	窗口2	卖出一张票, 剩余	37 张	
	窗口3	卖出一张票, 剩余	37 张	
	窗口4	卖出一张票, 剩余	37 张	
	窗口5	卖出一张票, 剩余	36 张	
	窗口1	卖出一张票, 剩余	36 张	
	窗口3	卖出一张票, 剩余	36 张	
	窗口2	卖出一张票, 剩余	36 张	
	窗口4	卖出一张票, 剩余	36 张	
	窗口5	卖出一张票, 剩余	35 张	
	窗口3	卖出一张票, 剩余	35 张	
	窗口1	卖出一张票, 剩余	35 张	
	窗口2	卖出一张票, 剩余	35 张	
	窗口4	卖出一张票, 剩余	35 张	
	窗口5	卖出一张票, 剩余	34 张	
	窗口1	卖出一张票, 剩余	34 张	
	窗口3	卖出一张票, 剩余	34 张	
	窗口4	卖出一张票, 剩余	34 张	
	窗口2	卖出一张票, 剩余	34 张	
	窗口1	卖出一张票, 剩余	33 张	
	窗口2	卖出一张票, 剩余	33 张	
	窗口3	卖出一张票, 剩余	33 张	
	窗口4	卖出一张票, 剩余	33 张	
	窗口5	卖出一张票, 剩余	33 张	
	窗口1	卖出一张票, 剩余	32 张	
	窗口4	卖出一张票, 剩余	32 张	
	窗口3	卖出一张票, 剩余	32 张	
	窗口2	卖出一张票, 剩余	32 张	
	窗口5	卖出一张票, 剩余	32 张	
	窗口1	卖出一张票, 剩余	31 张	
	窗口4	卖出一张票, 剩余	31 张	
	窗口3	卖出一张票, 剩余	31 张	
	窗口2	卖出一张票, 剩余	31 张	
	窗口5	卖出一张票, 剩余	31 张	
	窗口1	卖出一张票, 剩余	30 张	
	窗口3	卖出一张票, 剩余	30 张	
	窗口2	卖出一张票, 剩余	30 张	
	窗口4	卖出一张票, 剩余	30 张	
	窗口5	卖出一张票, 剩余	30 张	
	窗口1	卖出一张票, 剩余	29 张	 窗口5 卖出一张票, 剩余 29 张
	窗口4	卖出一张票, 剩余	29 张	
	窗口2	卖出一张票, 剩余	29 张	
	窗口3	卖出一张票, 剩余	29 张	
	窗口5	卖出一张票, 剩余	28 张	
	窗口4	卖出一张票, 剩余	28 张	
	窗口2	卖出一张票, 剩余	28 张	
	窗口1	卖出一张票, 剩余	28 张	
	窗口3	卖出一张票, 剩余	28 张	
	窗口2	卖出一张票, 剩余	27 张	
	窗口5	卖出一张票, 剩余	27 张	
	窗口3	卖出一张票, 剩余	27 张	
	窗口4	卖出一张票, 剩余	27 张	
	窗口1	卖出一张票, 剩余	27 张	
	窗口1	卖出一张票, 剩余	26 张	
	窗口4	卖出一张票, 剩余	26 张	

窗口2	卖出一张票,剩余	26 张
窗口5	卖出一张票,剩余	26 张
窗口3	卖出一张票,剩余	26 张
窗口4	卖出一张票,剩余	25 张
窗口2	卖出一张票,剩余	25 张
窗口5	卖出一张票,剩余	25 张
窗口1	卖出一张票,剩余	25 张
窗口3	卖出一张票,剩余	25 张
窗口2	卖出一张票,剩余	24 张
窗口4	卖出一张票,剩余	24 张
窗口1	卖出一张票,剩余	24 张
窗口5	卖出一张票,剩余	24 张
窗口3	卖出一张票,剩余	24 张
窗口2	卖出一张票,剩余	23 张
窗口4	卖出一张票,剩余	23 张
窗口5	卖出一张票,剩余	23 张
窗口1	卖出一张票,剩余	23 张
窗口3	卖出一张票,剩余	23 张
窗口4	卖出一张票,剩余	22 张
窗口3	卖出一张票,剩余	22 张
窗口1	卖出一张票,剩余	22 张
窗口5	卖出一张票,剩余	22 张
窗口2	卖出一张票,剩余	22 张
窗口4	卖出一张票,剩余	21 张
窗口3	卖出一张票,剩余	21 张
窗口5	卖出一张票,剩余	21 张
窗口1	卖出一张票,剩余	21 张
窗口2	卖出一张票,剩余	21 张
窗口3	卖出一张票,剩余	20 张
窗口4	卖出一张票,剩余	20 张
窗口5	卖出一张票,剩余	20 张
窗口2	卖出一张票,剩余	20 张
窗口1	卖出一张票,剩余	20 张
窗口4	卖出一张票,剩余	19 张
窗口3	卖出一张票,剩余	19 张
窗口2	卖出一张票,剩余	19 张
窗口1	卖出一张票,剩余	19 张
窗口5	卖出一张票,剩余	19 张
窗口4	卖出一张票,剩余	18 张
窗口1	卖出一张票,剩余	18 张
窗口2	卖出一张票,剩余	18 张
窗口3	卖出一张票,剩余	18 张
窗口5	卖出一张票,剩余	18 张
窗口4	卖出一张票,剩余	17 张
窗口1	卖出一张票,剩余	17 张
窗口3	卖出一张票,剩余	17 张
窗口5	卖出一张票,剩余	17 张
窗口2	卖出一张票,剩余	17 张
窗口4	卖出一张票,剩余	16 张
窗口1	卖出一张票,剩余	16 张
窗口3	卖出一张票,剩余	16 张
窗口2	卖出一张票,剩余	16 张
窗口5	卖出一张票,剩余	16 张
窗口4	卖出一张票,剩余	15 张
窗口1	卖出一张票,剩余	15 张
窗口2	卖出一张票,剩余	15 张
窗口3	卖出一张票,剩余	15 张
窗口5	卖出一张票,剩余	15 张
窗口4	卖出一张票,剩余	14 张
窗口1	卖出一张票,剩余	14 张

窗口3	卖出一张票, 剩余	14 张
窗口2	卖出一张票, 剩余	14 张
窗口5	卖出一张票, 剩余	14 张
窗口4	卖出一张票, 剩余	13 张
窗口2	卖出一张票, 剩余	13 张
窗口1	卖出一张票, 剩余	13 张
窗口5	卖出一张票, 剩余	13 张
窗口3	卖出一张票, 剩余	13 张
窗口4	卖出一张票, 剩余	12 张
窗口5	卖出一张票, 剩余	12 张
窗口1	卖出一张票, 剩余	12 张
窗口2	卖出一张票, 剩余	12 张
窗口3	卖出一张票, 剩余	12 张
窗口4	卖出一张票, 剩余	11 张
窗口2	卖出一张票, 剩余	11 张
窗口1	卖出一张票, 剩余	11 张
窗口3	卖出一张票, 剩余	11 张
窗口5	卖出一张票, 剩余	11 张
窗口4	卖出一张票, 剩余	10 张
窗口2	卖出一张票, 剩余	10 张
窗口1	卖出一张票, 剩余	10 张
窗口3	卖出一张票, 剩余	10 张
窗口5	卖出一张票, 剩余	10 张
窗口4	卖出一张票, 剩余	9 张
窗口2	卖出一张票, 剩余	9 张
窗口1	卖出一张票, 剩余	9 张
窗口3	卖出一张票, 剩余	9 张
窗口5	卖出一张票, 剩余	9 张
窗口4	卖出一张票, 剩余	8 张
窗口1	卖出一张票, 剩余	8 张
窗口2	卖出一张票, 剩余	8 张
窗口5	卖出一张票, 剩余	8 张
窗口3	卖出一张票, 剩余	8 张
窗口4	卖出一张票, 剩余	7 张
窗口1	卖出一张票, 剩余	7 张
窗口2	卖出一张票, 剩余	7 张
窗口5	卖出一张票, 剩余	7 张
窗口3	卖出一张票, 剩余	7 张
窗口4	卖出一张票, 剩余	6 张
窗口2	卖出一张票, 剩余	6 张
窗口1	卖出一张票, 剩余	6 张
窗口5	卖出一张票, 剩余	6 张
窗口3	卖出一张票, 剩余	6 张
窗口4	卖出一张票, 剩余	5 张
窗口2	卖出一张票, 剩余	5 张
窗口1	卖出一张票, 剩余	5 张
窗口3	卖出一张票, 剩余	5 张
窗口5	卖出一张票, 剩余	5 张
窗口4	卖出一张票, 剩余	4 张
窗口2	卖出一张票, 剩余	4 张
窗口1	卖出一张票, 剩余	4 张
窗口3	卖出一张票, 剩余	4 张
窗口5	卖出一张票, 剩余	4 张
窗口4	卖出一张票, 剩余	3 张
窗口2	卖出一张票, 剩余	3 张
窗口1	卖出一张票, 剩余	3 张
窗口3	卖出一张票, 剩余	3 张
窗口5	卖出一张票, 剩余	3 张
窗口4	卖出一张票, 剩余	2 张
窗口2	卖出一张票, 剩余	2 张

窗口1 卖出一张票,剩余 2 张
窗口3 卖出一张票,剩余 2 张
窗口5 卖出一张票,剩余 2 张
窗口4 卖出一张票,剩余 1 张
窗口1 卖出一张票,剩余 1 张
窗口2 卖出一张票,剩余 1 张
窗口3 卖出一张票,剩余 1 张
窗口5 卖出一张票,剩余 1 张
窗口1 卖出一张票,剩余 0 张
窗口1 没票了,停止售票
窗口2 卖出一张票,剩余 0 张
窗口2 没票了,停止售票
窗口4 卖出一张票,剩余 0 张
窗口4 没票了,停止售票
窗口3 卖出一张票,剩余 0 张
窗口3 没票了,停止售票
窗口5 卖出一张票,剩余 0 张
窗口5 没票了,停止售票

✱ 结果:最终剩余票数 = 0(期望 0,但出现负数 = 超卖!)

=====

✓ 加锁版本 -- 数据安全可靠

=====

窗口1 卖出一张票,剩余 49 张
窗口2 卖出一张票,剩余 48 张
窗口3 卖出一张票,剩余 47 张
窗口4 卖出一张票,剩余 46 张
窗口5 卖出一张票,剩余 45 张
窗口1 卖出一张票,剩余 44 张
窗口2 卖出一张票,剩余 43 张
窗口3 卖出一张票,剩余 42 张
窗口4 卖出一张票,剩余 41 张
窗口5 卖出一张票,剩余 40 张
窗口1 卖出一张票,剩余 39 张
窗口2 卖出一张票,剩余 38 张
窗口3 卖出一张票,剩余 37 张
窗口4 卖出一张票,剩余 36 张
窗口5 卖出一张票,剩余 35 张
窗口1 卖出一张票,剩余 34 张
窗口2 卖出一张票,剩余 33 张
窗口3 卖出一张票,剩余 32 张
窗口4 卖出一张票,剩余 31 张
窗口5 卖出一张票,剩余 30 张
窗口1 卖出一张票,剩余 29 张
窗口2 卖出一张票,剩余 28 张
窗口3 卖出一张票,剩余 27 张
窗口4 卖出一张票,剩余 26 张
窗口5 卖出一张票,剩余 25 张
窗口1 卖出一张票,剩余 24 张
窗口2 卖出一张票,剩余 23 张
窗口3 卖出一张票,剩余 22 张
窗口4 卖出一张票,剩余 21 张
窗口5 卖出一张票,剩余 20 张
窗口1 卖出一张票,剩余 19 张
窗口2 卖出一张票,剩余 18 张
窗口3 卖出一张票,剩余 17 张
窗口4 卖出一张票,剩余 16 张
窗口5 卖出一张票,剩余 15 张
窗口1 卖出一张票,剩余 14 张

```

窗口2 卖出一张票,剩余 13 张
窗口3 卖出一张票,剩余 12 张
窗口4 卖出一张票,剩余 11 张
窗口5 卖出一张票,剩余 10 张
窗口1 卖出一张票,剩余 9 张
窗口2 卖出一张票,剩余 8 张
窗口3 卖出一张票,剩余 7 张
窗口4 卖出一张票,剩余 6 张
窗口5 卖出一张票,剩余 5 张
窗口1 卖出一张票,剩余 4 张
窗口2 卖出一张票,剩余 3 张
窗口3 卖出一张票,剩余 2 张
窗口4 卖出一张票,剩余 1 张
窗口5 卖出一张票,剩余 0 张
窗口1 没票了,停止售票
窗口2 没票了,停止售票
窗口3 没票了,停止售票
窗口4 没票了,停止售票
窗口5 没票了,停止售票

```

🎉 结果:最终剩余票数 = 0(期望 0,完全正确!)

📄 总结

不加锁的后果:多个线程同时读到 `total_tickets=1`,
都认为还有票,结果都卖了出去,变成 `-1,-2...`
这就是典型的"超卖"问题。

解决办法:用 `with lock:` 把"查票"、"修改"包起来,
同一时刻只允许一个线程进入,保证数据安全。

```

In [19]: # =====
# Python 异常捕获示例 (try/except/finally)
# =====

# -----
# 1. 基本结构:try-except
# -----

print("1 基本 try-except")
print("-" * 40)

try:
    num = int(input("请输入一个数字:")) # 如果输入的不是数字...
    result = 10 / num # 如果输入的是 0...
    print(f"10 / {num} = {result}")
except ValueError:
    print("❌ 输入错误:请输入有效的数字!")
except ZeroDivisionError:
    print("❌ 数学错误:除数不能为 0!")
# 运行时等待输入,这里直接演示,用模拟数据
print(" (上方代码需交互输入,见下方自动演示)\n")

# -----
# 自动演示:用实际数据模拟各种异常
# -----

print("2 自动演示各种异常情况")
print("-" * 40)

```

```

test_cases = ["abc", "0", "2", ""]

for val in test_cases:
    print(f"\n 输入: '{val}'")
    try:
        num = int(val)
        result = 10 / num
        print(f"    ✓ 10 / {num} = {result}")
    except ValueError:
        print(f"    ✗ ValueError: '{val}' 不是数字!")
    except ZeroDivisionError:
        print(f"    ✗ ZeroDivisionError: 10 / 0 不允许!")
    except:
        print(f"    ✗ 其他未知错误!")

# -----
# 3. try-except-else-finally 完整结构
# -----
print("\n" + "📌 完整结构: try-except-else-finally")
print("-" * 40)

def divide_safe(a, b):
    """安全的除法函数"""
    try:
        result = a / b          # 尝试执行
    except ZeroDivisionError:    # 捕获特定异常
        print(f"    ✗ {a} / {b} -> 除数不能为 0")
        return None
    except TypeError:           # 捕获类型错误
        print(f"    ✗ {a} / {b} -> 类型错误")
        return None
    else:                       # 没有异常时执行
        print(f"    ✓ {a} / {b} = {result}")
        return result
    finally:                    # 不管有没有异常都执行
        print(f"    (finally:清理工作完成)")

print()
divide_safe(10, 2)
print()
divide_safe(10, 0)
print()
divide_safe(10, "a")

# -----
# 4. 捕获多个异常用元组
# -----
print("\n" + "📌 一次捕获多种异常")
print("-" * 40)

data_list = [100, 0, "hello", [1,2,3]]

for item in data_list:
    try:
        result = 100 / item
        print(f"    100 / {item} = {result}")
    except (ZeroDivisionError, TypeError, ValueError) as e:
        # 用元组把多种异常写在一起
        print(f"    100 / {item} -> 异常:{e}")

```

```

# -----
# 5. 主动抛出异常 raise
# -----
print("\n" + "5 主动抛出异常 raise")
print("-" * 40)

def check_age(age):
    """检查年龄是否合法"""
    if not isinstance(age, int):
        raise TypeError("年龄必须是整数")
    if age < 0:
        raise ValueError("年龄不能为负数")
    if age > 150:
        raise ValueError("年龄不合理")
    print(f"✅ 年龄 {age} 合法")

for age in [25, -5, 200, "abc"]:
    try:
        check_age(age)
    except (TypeError, ValueError) as e:
        print(f"❌ check_age({age}) -> {e}")

# -----
# 6. 自定义异常类
# -----
print("\n" + "6 自定义异常类")
print("-" * 40)

class BalanceNotEnoughError(Exception):
    """余额不足异常"""
    def __init__(self, balance, amount):
        self.balance = balance
        self.amount = amount
        super().__init__(f"余额不足:当前余额 {balance}元,需要 {amount}元")

def withdraw(balance, amount):
    """取款函数"""
    if amount > balance:
        raise BalanceNotEnoughError(balance, amount)
    new_balance = balance - amount
    print(f"✅ 取款 {amount}元成功,剩余 {new_balance}元")
    return new_balance

for amt in [100, 500, 50]:
    try:
        withdraw(200, amt)
    except BalanceNotEnoughError as e:
        print(f"❌ {e}")

# -----
# 小总结
# -----
print("\n" + "=" * 60)
print("📌 异常捕获小结")
print("=" * 60)
print("""
try:      尝试执行可能会出错的代码
except:   捕获并处理异常
else:     没有异常时执行(可选)

```

finally: 不管有没有异常都执行(可选)

常用异常类型:

<code>ValueError</code>	-> 值错误
<code>TypeError</code>	-> 类型错误
<code>ZeroDivisionError</code>	-> 除零错误
<code>FileNotFoundError</code>	-> 文件未找到
<code>KeyError</code>	-> 字典键不存在
<code>IndexError</code>	-> 列表索引越界

```
raise Exception("信息") # 主动抛出异常
"""
```

1 基本 try-except

✖ 输入错误，请输入有效的数字！
 <上方代码需交互输入，见下方自动演示>

2 自动演示各种异常情况

输入: 'abc'

✖ ValueError: 'abc' 不是数字!

输入: '0'

✖ ZeroDivisionError: 10 / 0 不允许!

输入: '2'

✔ 10 / 2 = 5.0

输入: ''

✖ ValueError: '' 不是数字!

3 完整结构: try-except-else-finally

✔ 10 / 2 = 5.0
 (finally:清理工作完成)

✖ 10 / 0 -> 除数不能为 0
 (finally:清理工作完成)

✖ 10 / a -> 类型错误
 (finally:清理工作完成)

4 一次捕获多种异常

100 / 100 = 1.0
 100 / 0 -> 异常:division by zero
 100 / hello -> 异常:unsupported operand type(s) for /: 'int' and 'str'
 100 / [1, 2, 3] -> 异常:unsupported operand type(s) for /: 'int' and 'list'

5 主动抛出异常 raise

✔ 年龄 25 合法
 ✖ check_age(-5) -> 年龄不能为负数
 ✖ check_age(200) -> 年龄不合理
 ✖ check_age(abc) -> 年龄必须是整数

6 自定义异常类

✔ 取款 100元成功,剩余 100元
 ✖ 余额不足:当前余额 200元,需要 500元
 ✔ 取款 50元成功,剩余 150元

异常捕获小结

try: 尝试执行可能会出错的代码
 except: 捕获并处理异常
 else: 没有异常时执行(可选)
 finally: 不管有没有异常都执行(可选)

常用异常类型:

ValueError -> 值错误
 TypeError -> 类型错误
 ZeroDivisionError -> 除零错误
 FileNotFoundError -> 文件未找到
 KeyError -> 字典键不存在
 IndexError -> 列表索引越界

raise Exception("信息") # 主动抛出异常

```
In [ ]: import os
import shutil

dataset_dir = r"D:\project\step1\week3\dataset"
categories = {"dr": "drink", "ph": "phone"}
default_category = "smoke"
sub_dirs = ["train", "val"]

def classify_filename(filename):
    """根据文件名判断类别"""
    name, ext = os.path.splitext(filename)
    if ext.lower() not in [".jpg", ".jpeg", ".png", ".bmp"]:
        return None
    for prefix, category in categories.items():
        if name.startswith(prefix):
            return category
    return default_category

def organize_directory(base_dir, sub_dir):
    """整理指定目录下的图片到分类子目录"""
    source_path = os.path.join(base_dir, sub_dir)
    if not os.path.isdir(source_path):
        print(f" ⚠️ 目录不存在: {source_path}")
        return

    files = [f for f in os.listdir(source_path)
              if os.path.isfile(os.path.join(source_path, f))]
    if not files:
        print(f" 📁 {sub_dir}/: 无需整理")
        return

    print(f"\n📁 整理 {sub_dir}/ ...")
    moved = 0
    for filename in sorted(files):
        cat = classify_filename(filename)
        if cat is None:
            continue
        target_dir = os.path.join(source_path, cat)
        os.makedirs(target_dir, exist_ok=True)
        dst = os.path.join(target_dir, filename)
        if os.path.exists(dst):
            continue
        shutil.move(os.path.join(source_path, filename), dst)
        moved += 1
    print(f" ✅ 移动了 {moved} 个文件")
```

```

# =====
# 执行整理
# =====
print("🚀 开始整理数据集图片\n分类规则:")
for p, c in categories.items():
    print(f"  {p}*.jpg -> {c}")
print(f"  其他      -> {default_category}")

for sd in sub_dirs:
    organize_directory(dataset_dir, sd)

# 打印结果
print("\n" + "=" * 50)
print("📁 整理结果")
print("=" * 50)
for sd in sub_dirs:
    path = os.path.join(dataset_dir, sd)
    if not os.path.isdir(path):
        continue
    print(f"\n{sd}/")
    for cat in ["smoke", "drink", "phone"]:
        cp = os.path.join(path, cat)
        if os.path.isdir(cp):
            n = len([f for f in os.listdir(cp) if os.path.isfile(os.path.join(cp, f))])
            print(f"  └─ {cat}/ ({n} 个文件)")

print(f"\n✅ 整理完成!")

```

🚀 开始整理数据集图片
分类规则：
dr*.jpg -> drink
ph*.jpg -> phone
其他 -> smoke

📁 整理 train/ ...
✅ 移动了 35 个文件

📁 整理 val/ ...
✅ 移动了 26 个文件

=====

📁 整理结果

=====

```

train/
├─ smoke/ (8 个文件)
├─ drink/ (9 个文件)
└─ phone/ (18 个文件)

```

```

val/
├─ smoke/ (8 个文件)
├─ drink/ (9 个文件)
└─ phone/ (9 个文件)

```

✅ 整理完成!